

TransitFlow AI: Master Development Roadmap

Strategy & Execution Architecture

1. Competition Submission Timeline & Phase Alignment

The development roadmap is directly aligned with the official **MAPID WebGIS Competition 2026** schedule:

- **Phase 1: Proposal Submission (June 24 – August 2, 2026):** Submission of the technical proposal document detailing spatial problem statements, field survey methodology, MAPID product integration, and zero-cost free-tier architecture.
- **Top 50 Curation Announcement (August 5, 2026):** Unlocks MAPID Top 50 Perks (MAPID Form SDK, SINI AI platform API, GEO MAPID licenses, GEO DATA catalogs, and survey activity budget).
- **Phase 2: MVP WebGIS Development Window (August 7 – September 14, 2026):** Rapid execution of Sprints 1 through 5, producing a fully deployed public WebGIS application (`transitflow.id`), PRD document, spatial metadata catalog, and field survey documentation for final judging.
- **Top 10 Finalist Showcase (September 2026):** Sprints 6 through 12 form the long-term enterprise B2G commercialization roadmap for nationwide municipal transit adoption.

2. The "100% Incrementally Submit-able" Mindset

Is every single sprint genuinely submit-able on its own? Yes, absolutely.

The roadmap is engineered under a strict **Cumulative Value & Instant-Submit Guarantee**:

- **Zero Half-Baked Features:** Every single sprint yields a complete, self-contained, live-deployed product increment.
- **Deadline Resilience:** If development is frozen or cut short at *any* sprint due to hackathon time constraints, the project is **never broken or incomplete**. The team can immediately submit the live application URL at whichever sprint was completed:
 - **Stopped at Sprint 1:** A fully functional interactive Station Exit & Obstacle Spatial Mapping WebGIS.
 - **Stopped at Sprint 2:** An AI-powered WebGIS converting survey photos/audio into spatial metadata via MAPID SINI AI & Gemini 3.6 Flash.
 - **Stopped at Sprint 3:** A real-time overcrowd risk engine broadcasting live VCI scores & automated Telegram alerts.
 - **Stopped at Sprint 4:** An active spatial management tool offering 15-minute ojol buffer zones & interactive barrier simulation.
 - **Stopped at Sprint 5:** A complete monsoon flood resilience WebGIS with public commuter PWA safe-path navigation.
- **Cumulative Engineering:** Each sprint builds additively on top of previous layers without refactoring or throwaway code.

Sprint 1: Core Field Survey & Spatial Node Registry

- **Deliverable:** Interactive Station Exit & Obstacle Mapping WebGIS.
- **Key Features:**
 - MAPID Form integration for field surveyors to map station exit doors, stairs, sidewalk widths, and physical obstacles.
 - Digital Spatial Registry of transit hubs (Dukuh Atas, Manggarai, Tanah Abang) rendered on GEO MAPID vector tiles.
 - Basic spatial point & polygon layer editor.
- **Why this Sprint?:** Without a high-precision spatial baseline of physical station exit geometries, no AI model or capacity calculation can exist.
- **Tech Stack & Infrastructure:**
 - **Frontend:** React 19 + Vite / Next.js 16, MapLibre GL JS (integrated into GEO MAPID layer renderer), Tailwind CSS.
 - **Backend:** Node.js (TypeScript) + Express / Hono microservice, Supabase PostgreSQL + PostGIS (spatial database).
 - **MAPID Ecosystem:** MAPID Form SDK (Top 50 Perk), GEO MAPID API (Top 50 Perk).
- **Tools & Procurement (100% Free Tier):**
 - **MAPID Free Tier / Top 50 Perk** (For GEO MAPID layer publishing & Form API access).
 - **Supabase Free Tier (PostGIS Enabled)** (500 MB DB + 50,000 MAU free).
 - **Vercel / Cloudflare Pages Free Tier** hosting for web GIS portal.
- **Step-by-Step Implementation Procedure (10 Steps):**
 1. **Initialize Monorepo Workspace:** Set up Next.js 16 (App Router) + React 19 frontend and Node.js (TypeScript) / Express / Hono backend with pnpm workspace.
 2. **Provision Supabase Spatial DB:** Create free Supabase project, enable postgis extension, and execute schema migrations for stations, exit_doors, sidewalks, and obstacles tables.
 3. **Configure MAPID Form Account:** Activate MAPID Top 50 perk on MAPID Form, building custom survey forms for exit door dimensions, stairs, sidewalk widths, and physical obstacle markers.
 4. **Integrate MAPID Form Webhook/API:** Implement backend webhook receiver to auto-ingest completed MAPID Form field survey entries directly into Supabase exit_doors PostGIS tables.
 5. **Set Up MapLibre GL JS Base Map:** Embed MapLibre GL JS in Next.js 16 WebGIS UI with custom vector tile style sources from GEO MAPID API.
 6. **Build Spatial Point & Polygon Editor:** Develop interactive UI controls using MapLibre draw tools to map station exit footprints, staircases, and sidewalk boundaries.
 7. **Implement GeoJSON Spatial Exporter:** Build REST API endpoints to export and sync mapped station exit geometries as validated GeoJSON datasets to GEO MAPID.
 8. **Map Initial Target Hubs:** Conduct initial field surveys via MAPID Form at Stasiun Dukuh Atas, Manggarai, and Tanah Abang to establish baseline spatial data.
 9. **Implement Basic Access Control:** Integrate Supabase Auth / Clerk Free Tier to restrict spatial map editing features to authorized field surveyors.
 10. **Deploy Sprint 1 Base:** Deploy frontend WebGIS to Vercel Free Tier and backend service to Cloudflare Pages / Render Free Tier, verifying real-time vector tile rendering.

Sprint 2: MAPID SINI AI Unstructured Data Parsing Engine

- **Deliverable:** Automated Field Photo & Audio Note Parser.
- **Key Features:**
 - Image AI pipeline: Automatically extracts crowd density counts, informal angkot queue numbers, and street vendor footprints from field survey photos.
 - Voice NLP pipeline: Converts surveyor audio recordings ("Angkot double parking on Gate 2") into structured hazard attributes.
 - Auto-attaches AI-extracted metadata to Sprint 1's station spatial nodes.
- **Why this Sprint?:** Leverages Sprint 1's spatial registry to start ingesting unstructured data—satisfying MAPID's core technical requirement early.
- **Tech Stack & Infrastructure:**
 - **AI / Computer Vision:** Python (FastAPI), PyTorch / OpenCV, YOLO26 (for state-of-the-art NMS-free object/crowd detection), Vision Transformer (ViT).
 - **Audio / NLP:** Google Gemini 3.6 Flash Free Tier (Multimodal Vision + Structured JSON extraction) + Whisper Local (Open Source Audio-to-Text).
 - **Storage:** Supabase Storage / Cloudflare R2 Free Tier for survey photo and audio storage.
- **Tools & Procurement (100% Free Tier):**
 - **MAPID SINI AI Platform API / Top 50 Perk.**
 - **Google Gemini 3.6 Flash Free Tier API** (15 RPM / 1M Tokens/min free allowance).
 - **Whisper Local (Open Source)** running locally (Rp 0 cost).
 - **Cloudflare R2 / Supabase Storage Free Tier** (10 GB free media storage).
- **Step-by-Step Implementation Procedure (10 Steps):**
 1. **Set Up Python FastAPI AI Microservice:** Create a Python FastAPI microservice dedicated to executing local computer vision and NLP parsing pipelines.
 2. **Integrate Google Gemini 3.6 Flash API:** Configure Gemini 3.6 Flash SDK (Free Tier) with strict JSON response_schema enforcement for multimodal vision analysis.
 3. **Set Up Whisper Local Pipeline:** Install open-source faster-whisper locally to convert surveyor audio notes into Bahasa Indonesia text.
 4. **Deploy YOLO26 Crowd & Obstacle Detection:** Initialize YOLO26 (Ultralytics NMS-free model) on survey photos to extract pedestrian counts, informal vendor footprints, and angkot queues.
 5. **Configure Cloudflare R2 / Supabase Storage:** Provision free tier object storage bucket for survey media asset uploads.
 6. **Connect MAPID SINI AI Webhook Pipeline:** Trigger FastAPI AI pipeline automatically upon receiving raw photo/audio uploads from MAPID Form.
 7. **Build Structured Attribute Extractor:** Process Gemini 3.6 Flash and YOLO26 outputs into structured JSON attributes: pedestrian_count, angkot_queue_length, vendor_blockage_pct.
 8. **Link AI Metadata to Spatial Geometry:** Update Supabase PostGIS exit_doors records by merging extracted AI obstacle metadata into spatial node attributes.
 9. **Build AI Ingestion QA Dashboard:** Develop a Next.js 16 review UI displaying side-by-side survey photos, YOLO26 bounding boxes, Gemini JSON attributes, and manual edit controls.
 10. **End-to-End Validation:** Test complete pipeline from taking a field photo/audio on MAPID Form to rendering extracted spatial hazard metadata in <30 seconds.

Sprint 3: Real-Time Volumetric Choke Index (VCI) Engine

- **Deliverable:** Live Overcrowding Heatmap & Risk Scoring Engine.
- **Key Features:**
 - Mathematical Volumetric Choke Index (VCI) algorithm:

$$VCI = \min \left(100, \frac{\text{Pedestrian Flow Rate} + \text{Vehicular Drop-off Surge}}{\text{Effective Channel Width} \times \text{Walkway Compliance Factor } (\alpha)} \times 100 \right)$$

- GEO MAPID real-time color-coded exit risk layer (Green = Smooth, Yellow = Warning, Red = Choke Risk).
 - Automated threshold alerts sent via email/Telegram to hub station masters when $VCI \geq 80$.
- **Why this Sprint?:** Combines Sprint 1's physical widths with Sprint 2's AI crowd counts to generate the core Volumetric Choke Index.
- **Tech Stack & Infrastructure:**
 - **Backend & Stream Processing:** Upstash Redis (Pub/Sub & in-memory caching), Node.js worker queues (BullMQ).
 - **Spatial Analytics:** Turf.js (in-browser spatial analysis) + PostGIS spatial functions (ST_Buffer, ST_Intersects).
 - **Alert Messaging:** Telegram Bot API, Discord Webhooks, SendGrid Free Tier (Email API).
- **Tools & Procurement (100% Free Tier):**
 - **Upstash Redis Free Tier** (10,000 requests/day free).
 - **Telegram Bot API & Discord Webhooks** (100% Free unlimited integration).
 - **SendGrid Free Tier** (100 emails/day free).
- **Step-by-Step Implementation Procedure (10 Steps):**
 1. **Implement VCI Mathematical Engine:** Code the Volumetric Choke Index mathematical formula in TypeScript, incorporating dynamic compliance factor α .
 2. **Provision Upstash Redis Instance:** Set up Upstash Redis Free Tier (10,000 req/day) for high-speed in-memory spatial caching of live VCI scores.
 3. **Build PostGIS Spatial Buffer Functions:** Write SQL functions (ST_Buffer, ST_Intersects) to query active choke radii within 150m of station exits.
 4. **Implement Real-time VCI Heatmap Layer:** Build dynamic GeoJSON vector tile styling in MapLibre GL JS (Green: $VCI < 50$, Yellow: 50-79, Red: $VCI \geq 80$).
 5. **Set Up Telegram Bot Alert Channel:** Register a Telegram Bot via BotFather and write an automated alert dispatcher for hub station masters.
 6. **Set Up Discord & SendGrid Free Webhooks:** Implement backup alert channels using Discord Webhooks and SendGrid Free Tier for automated surge email alerts.
 7. **Build Background Queue Worker:** Deploy BullMQ / Upstash Redis background job runner to re-calculate station exit VCI every 60 seconds.
 8. **Trigger Automated Surge Notifications:** Configure threshold trigger ($VCI \geq 80$) to broadcast exit surge alerts with location maps to station masters in < 5 seconds.
 9. **Build Station Operator Alert Banner:** Add live flashing alert banners and audio cues to the Next.js 16 station master dashboard.
 10. **Stress Test VCI Engine:** Simulate peak evening rush hour surge payloads across Manggarai and Dukuh Atas to verify zero-latency Redis cache reads.

Sprint 4: Dynamic Curb & Micro-Mobility Buffer Allocator

- **Deliverable:** Dynamic Pick-Up Zone & Barrier Allocation Engine.
- **Key Features:**
 - Algorithmic space re-allocator: Generates temporary 15-minute ojek (online ride-hailing) pick-up zones during surge hours.
 - Recommends physical barrier placement coordinates (e.g., stanchion dividers) to maintain a 2-meter clear pedestrian walking lane.
 - Interactive spatial simulator on GEO MAPID allowing station operators to toggle barrier configurations.
- **Why this Sprint?:** Moves from passive risk scoring (Sprint 3) to active spatial solutions (re-allocating curb space and pedestrian channels).
- **Tech Stack & Infrastructure:**
 - **Simulation Engine:** Turf.js, Voronoi Diagram generators, NetworkX / OSRM (Open Source Routing Machine on Render Free Tier).
 - **Frontend UI:** Konva.js / Canvas API overlay on MAPID vector tiles for drag-and-drop barrier simulation.
- **Tools & Procurement (100% Free Tier):**
 - **OSRM Server** running on Render Free Tier / local container.
 - **OpenStreetMap (OSM) Jabodetabek road network data** (100% Free open data).
- **Step-by-Step Implementation Procedure (10 Steps):**
 1. **Download OSM Road Network Data:** Ingest OpenStreetMap (OSM) road network dumps for Jabodetabek into PostGIS using osm2pgrouting.
 2. **Deploy OSRM Server on Render Free Tier:** Host Open Source Routing Machine (OSRM) on Render Free Tier for pedestrian and micro-mobility network routing.
 3. **Develop Ojol Buffer Allocation Algorithm:** Write spatial algorithm to calculate dynamic 15-minute ojek drop-off zone coordinates away from main exit doors.
 4. **Develop Pedestrian Channel Dividers Logic:** Calculate required physical barrier placement coordinates to preserve a 2-meter clear pedestrian walking corridor.
 5. **Build Konva.js / Canvas Interactive Overlay:** Integrate Konva.js canvas layer on top of GEO MAPID vector tiles for drag-and-drop barrier placement simulation.
 6. **Create Barrier Toggle Simulator:** Allow station operators to click and toggle virtual stanchions/barriers and observe simulated VCI drop in real-time.
 7. **Build Dynamic Curb Slot Dispatcher:** Generate 15-minute slot reservations for ride-hailing pick-up zones.
 8. **Export Operational Dispatch Maps:** Build automated SVG/PDF export of temporary barrier layout plans for field traffic wardens.
 9. **Build Webhook Event Emitter for Ojol APIs:** Create Webhook endpoints (/api/v1/buffer-zones/active) formatted for Grab/Gojek API ingestion.
 10. **Conduct Field Simulation Test:** Test interactive barrier re-allocation UI with field operators at Dukuh Atas to measure simulated crowd throughput increase.

Sprint 5: Weather-Triggered Weather & Inundation Rerouting

- **Deliverable:** Monsoon Flood & Rain-Surge Resilience Module.
- **Key Features:**
 - Real-time weather API integration + field flood photo depth detection (via SINI AI / Gemini 3.6 Flash).
 - Dynamic pedestrian rain-detour routing (redirecting commuters away from flooded underpasses to covered walkways).
 - Integration with MAPID Elevation & Disaster Data catalogs.
- **Why this Sprint?:** Rain is the single largest external multiplier of transit exit paralysis in Indonesian cities.
- **Tech Stack & Infrastructure:**
 - **Weather Data:** BMKG Open Data API (Indonesian Meteorological Agency) / OpenWeatherMap Free Tier.
 - **Spatial DEM:** USGS / Copernicus 30m Open Digital Elevation Model (DEM), MAPID Disaster Data Catalog (Top 50 Perk).
 - **Routing Algorithm:** Dijkstra / A* algorithm with dynamic edge weight adjustments based on flood depth.
- **Tools & Procurement (100% Free Tier):**
 - **BMKG Open Data API / OpenWeatherMap Free Tier** (1,000 req/day free).
 - **MAPID Geo Data Disaster Catalog** (Provided free under MAPID Top 50 Perk).
- **Step-by-Step Implementation Procedure (10 Steps):**
 1. **Integrate BMKG Open Data Weather API:** Connect to BMKG (Indonesian Meteorological Agency) open weather API for real-time rain/storm alerts.
 2. **Integrate OpenWeatherMap Free API:** Set up OpenWeatherMap Free Tier (1,000 req/day) as secondary weather fallback feed.
 3. **Ingest MAPID Disaster & Elevation Catalogs:** Load MAPID Geo Data Disaster Catalog and USGS/Copernicus 30m DEM into PostGIS spatial database.
 4. **Build Flood Photo Depth Extractor:** Extend Gemini 3.6 Flash vision prompts to estimate water flood depth (cm) from field survey photos.
 5. **Develop Dynamic Pedestrian Dijkstra/A* Router:** Modify graph routing edge weights based on real-time flood depth (e.g., set weight to infinity for flooded underpasses).
 6. **Generate Indoor/Covered Detour Routes:** Map elevated covered walkways and indoor station corridors as rain-resilient detour edges.
 7. **Build Weather-Triggered Reroute Switch:** Automatically trigger rain-detour mode in VCI calculation when BMKG reports rainfall >20 mm/hr.
 8. **Add Rain Overlays on GEO MAPID Map:** Render live BMKG radar imagery and flooded underpass markers on MapLibre GL JS base map.
 9. **Build Public Rain-Path Recommendation Component:** Add "Rain Safe-Path" toggle to the commuter WebGIS UI.
 10. **Validate Monsoon Simulation:** Run simulated 50mm heavy rain scenario at Tanah Abang to verify automatic rerouting around flooded underpasses.

Sprint 6: Public Commuter Alert & Safe-Path WebGIS Portal

- **Deliverable:** Commuter-Facing Mobile WebGIS Application.
- **Key Features:**
 - Lightweight public WebGIS showing real-time station exit congestion levels.
 - Safe-Path Finder: Recommends the least congested station exit door for arriving train passengers.
 - Commuter crowdsourcing form on MAPID Form to report sudden exit blockages.
- **Why this Sprint?:** Expands the platform from internal station management to empowering the public commuter, driving user feedback.
- **Tech Stack & Infrastructure:**
 - **Frontend:** Progressive Web App (PWA) built with Next.js 16, React 19, Tailwind CSS, Workbox (offline caching).
 - **CDN & Edge Delivery:** Cloudflare Pages / Vercel Free Tier Network.
- **Tools & Procurement (100% Free Tier):**
 - **Cloudflare Pages / Vercel Free Tier** (Unlimited bandwidth free, edge CDN & DDoS protection included).
- **Step-by-Step Implementation Procedure (10 Steps):**
 1. **Initialize Public PWA Architecture:** Build responsive mobile-first Progressive Web App (PWA) using Next.js 16, React 19, and Workbox.
 2. **Configure Cloudflare Pages Free Hosting:** Deploy commuter WebGIS (transitflow.id) on Cloudflare Pages Free Tier with global CDN caching.
 3. **Implement Geo-Location Nearest Station Lookup:** Use HTML5 Geolocation API to auto-detect commuter position and load nearest transit exit status.
 4. **Build Safe-Path Door Finder:** Create intuitive UI guiding arriving train passengers to the least congested exit door (e.g., "Exit Door C is 40% clearer than Exit Door B").
 5. **Embed Lightweight MAPID Form Crowdsourcing:** Add quick 1-tap commuter report form via MAPID Form ("Report sudden blockage / broken escalator").
 6. **Implement Offline Service Worker Caching:** Cache station floorplans, exit vector maps, and safe-path logic for offline use inside subway tunnels.
 7. **Add Mobile Web Push Notifications:** Implement Web Push API for commuters to subscribe to real-time delay/surge alerts for their daily commute station.
 8. **Optimize Web Vitals & LCP:** Compress vector tiles, enable HTTP/3 on Cloudflare Pages, and achieve LCP < 1.2s on standard 4G mobile connections.
 9. **Perform Cross-Device Accessibility Audit:** Ensure WCAG 2.1 compliance, high-contrast UI mode, and screen-reader support for visually impaired commuters.
 10. **Launch Public Beta:** Roll out public PWA to pilot commuter testing group at Manggarai and Dukuh Atas hubs.

Sprint 7: Multi-Agency Dishub & Transit Command Center

- **Deliverable:** Multi-Tenant Enterprise Command Dashboard.
- **Key Features:**
 - Role-based access for Municipal Transportation Authorities (Dishub), Police, and Transit Operators (LRT/MRT/KAI).
 - Cross-agency dispatch workflow: One-click notification to dispatch traffic wardens to specific double-parking coordinates.
 - Analytics dashboard tracking historical bottleneck trends and response lead times.
- **Why this Sprint?:** Escalates the product to an enterprise multi-agency B2G tool once individual station tools (Sprint 1-6) are mature.
- **Tech Stack & Infrastructure:**
 - **Auth & Multi-Tenancy:** Clerk Free Tier / Supabase Auth (Multi-organization support, RBAC up to 10,000 MAU free).
 - **WebSockets:** Socket.io hosted on Render Free Tier / Upstash Redis PubSub (for instant screen syncing and live dispatch maps).
 - **Analytics Engine:** Supabase PostGIS + DuckDB (embedded fast spatial analytics) + Recharts / Chart.js.
- **Tools & Procurement (100% Free Tier):**
 - **Clerk / Supabase Auth Free Tier** (10,000 monthly active users free).
 - **Socket.io on Render Free Tier** (750 free instance hours/month).
 - **Supabase PostGIS Free Tier** for spatial logs & analytics.
- **Step-by-Step Implementation Procedure (10 Steps):**
 1. **Set Up Multi-Tenant RBAC via Clerk / Supabase Auth:** Configure Clerk / Supabase Auth Free Tier with multi-organization roles (Dishub, KAI, MRT, Police).
 2. **Build Enterprise Multi-Monitor Layout:** Design desktop dashboard using CSS Grid and Tailwind CSS optimized for multi-screen War Room display walls.
 3. **Deploy Socket.io Signaling Server on Render:** Host Socket.io server on Render Free Tier connected to Upstash Redis Pub/Sub for cross-agency state sync.
 4. **Build One-Click Warden Dispatch Workflow:** Enable Dishub operators to click map choke coordinates and instantly dispatch nearest field warden.
 5. **Integrate Embedded DuckDB Analytics:** Embed DuckDB in Node.js backend for ultra-fast in-memory aggregation of historical bottleneck logs.
 6. **Build Historical Bottleneck Heatmaps:** Create time-slider controls allowing operators to view exit choke patterns across past days, weeks, and months.
 7. **Build Multi-Agency Screen Syncing:** Implement Socket.io state sync so actions taken by Dishub War Room immediately reflect on Station Master screens.
 8. **Develop Cross-Agency SLA Tracker:** Track metrics on alert response lead times (e.g., time elapsed between VCI warning and warden arrival).
 9. **Implement Executive CSV/Excel Report Exporter:** Build automated daily operational report exporter for Ministry of Transportation (Kemenhub) executives.
 10. **Deploy Command Center Staging:** Conduct live simulation with municipal transit authorities to validate multi-tenant access control and dispatch speed.

Sprint 8: Predictive Crowd Surge & Event Simulation

- **Deliverable:** AI Event & Timetable Scenario Simulator.
- **Key Features:**
 - Predicts exit choke risks up to 48 hours in advance based on train schedule changes, concert/stadium events, and holiday commuter surges.
 - "What-If" scenario builder on GEO MAPID (e.g., "What happens to Exit 3 if Train Line 2 delays by 20 minutes?").
- **Why this Sprint?:** Shifts from real-time reaction (Sprint 3) to predictive forward-looking simulation.
- **Tech Stack & Infrastructure:**
 - **Time-Series ML Pipeline:** Python, XGBoost / Prophet / LightGBM, Pandas, Scikit-learn.
 - **Event Data Integration:** Web Scrapers (Playwright / Puppeteer) + Open Public Event Feeds / GTFS-Realtime public transit feeds.
- **Tools & Procurement (100% Free Tier):**
 - **Google Colab / Kaggle Free GPU Notebooks** for batch ML model training.
 - **GTFS Public Feeds / KAI Commuter & MRT Open Data** (100% Free).
- **Step-by-Step Implementation Procedure (10 Steps):**
 1. **Set Up Time-Series ML Environment:** Configure Python machine learning environment with XGBoost, LightGBM, Prophet, and Scikit-learn.
 2. **Build GTFS Transit Timetable Parser:** Write parser to ingest static and GTFS-Realtime timetable feeds from KAI Commuter, MRT, and LRT Jakarta.
 3. **Scrape Public Event Feeds:** Build Playwright / Puppeteer scrapers to harvest concert, stadium event, and holiday schedules in Jakarta.
 4. **Train 48-Hour Forward Predictive Model:** Train XGBoost model using historical VCI logs, train arrival schedules, event calendars, and weather forecasts.
 5. **Build "What-If" Scenario Builder UI:** Create interactive control panel on GEO MAPID to simulate scenarios (e.g., "Train delay + Stadium concert end at 21:00").
 6. **Set Up Google Colab / Kaggle Free GPU Pipeline:** Configure automated batch retraining pipeline using Google Colab / Kaggle Free GPUs.
 7. **Integrate Predictive Risk Layer into MapLibre:** Render 48-hour forward predictive VCI risk curves alongside real-time live scores.
 8. **Implement Early Warning Dispatch Trigger:** Send automated 24-hour advance warning notifications to station managers prior to major predicted surges.
 9. **Validate Predictive Accuracy:** Compare predicted VCI scores against actual field VCI logs at GBK / Senayan MRT station during major concert events.
 10. **Deploy Simulation Engine:** Package trained predictive pipeline as FastAPI endpoint integrated into the Sprint 7 Command Center.

Sprint 9: Automated CCTV Camera & IoT Sensor Data Pipeline

- **Deliverable:** Automated Real-Time Camera Stream Processing Module.
- **Key Features:**
 - Connects existing station CCTV streams directly to SINI AI / Gemini 3.6 Flash vision models (supplementing manual field survey photos).
 - Automated IoT foot-traffic counter integration.
 - Continuous background VCI score updating without human intervention.
- **Why this Sprint?:** Automates data ingestion after the manual field survey baseline (Sprint 1-2) is battle-tested.
- **Tech Stack & Infrastructure:**
 - **Video Streaming & Processing:** RTSP / WebRTC stream ingester, OpenCV, YOLO26-Pose / DeepSORT (pedestrian tracking).
 - **IoT Protocol:** Mosquitto MQTT Broker (Open Source) / EMQX Serverless Free Tier.
- **Tools & Procurement (100% Free Tier):**
 - **Self-hosted Mosquitto MQTT / EMQX Free Tier** (1,000 concurrent IoT connections free).
 - **Local edge worker / Google Gemini 3.6 Flash Free Tier** for periodic RTSP frame inference.
- **Step-by-Step Implementation Procedure (10 Steps):**
 1. **Set Up Open-Source Mosquitto MQTT Broker:** Deploy Mosquitto MQTT broker / EMQX Free Tier for lightweight IoT pedestrian counter data streaming.
 2. **Build RTSP / WebRTC Video Stream Ingestor:** Develop OpenCV / FFmpeg RTSP video stream receiver for existing station CCTV feeds.
 3. **Deploy YOLO26-Pose & DeepSORT:** Implement YOLO26-Pose and DeepSORT tracking pipeline in Python for real-time directional pedestrian counting.
 4. **Integrate Periodic CCTV Frame Sampling:** Extract 1 frame per 10 seconds from CCTV streams and feed to Google Gemini 3.6 Flash Free Tier for validation.
 5. **Build IoT Foot-Traffic Counter Adapter:** Write MQTT payload parser to convert raw hardware sensor counts into standardized VCI flow rate inputs.
 6. **Construct Automated Pipeline Failure Fallback:** Implement auto-fallback to field survey data (Sprint 2) if CCTV RTSP stream disconnects.
 7. **Implement Privacy-Preserving Frame Anonymization:** Apply real-time Gaussian blurring on all pedestrian faces and vehicle license plates before storage.
 8. **Build Live Stream Overlay Player:** Integrate WebRTC video player into Station Master dashboard with bounding box overlays.
 9. **Optimize Edge Inference Resource Allocation:** Cap edge worker processing to maintain zero cloud compute cost while processing 10 continuous streams.
 10. **24-Hour Continuous Run Test:** Perform 24-hour non-stop automated ingestion test at Manggarai Station CCTV feeds without human intervention.

Sprint 10: Non-Farebox Station Retail Micro-Kiosk Engine

- **Deliverable:** Commercial Retail & Vendor Spatial Zoning Module.
- **Key Features:**
 - Identifies "Zero-Choke Retail Zones" where informal vendors can operate legally without obstructing pedestrian flow.
 - Generates revenue potential scores for station kiosks based on historical commuter foot-traffic density.
 - MAPID SES (Socioeconomic Status) data overlay for commercial leasing teams.
- **Why this Sprint?:** Introduces a B2B monetization engine for transit operators seeking non-farebox revenue (NFR).
- **Tech Stack & Infrastructure:**
 - **Spatial Micro-Economics Engine:** Python (GeoPandas, Shapely), OpenStreetMap POI Dataset, MAPID SES Dataset (Top 50 Perk).
 - **PDF / Proposal Generator:** React PDF / Puppeteer export engine for commercial leasing proposals.
- **Tools & Procurement (100% Free Tier):**
 - **MAPID Geo Data Socioeconomic Status (SES) & POI Data catalog** (Provided free under MAPID Top 50 Perk).
- **Step-by-Step Implementation Procedure (10 Steps):**
 1. **Ingest OpenStreetMap POI Datasets:** Load spatial POI (Points of Interest) datasets around transit hubs into PostGIS.
 2. **Integrate MAPID Geo Data SES Catalog:** Overlay MAPID Socioeconomic Status (SES) and income proxy datasets (Top 50 Perk).
 3. **Develop "Zero-Choke Zoning" Algorithm:** Write spatial buffer algorithm identifying high foot-traffic areas located strictly outside VCI choke corridors.
 4. **Build Kiosk Foot-Traffic Heatmap:** Calculate commercial visibility scores for 3x3m micro-kiosk locations based on historical pedestrian flows.
 5. **Develop Retail Revenue Potential Estimator:** Create formula estimating monthly vendor sales potential based on SES demographic overlay and foot-traffic volume.
 6. **Build Interactive Kiosk Zoning UI:** Develop GEO MAPID layer allowing transit leasing teams to place, resize, and score virtual retail kiosks.
 7. **Integrate React PDF Proposal Generator:** Build automated 1-click PDF commercial leasing proposal generator with spatial site maps.
 8. **Enforce Pedestrian Flow Safety Constraints:** Automatically block kiosk placement if proposed location reduces clear walkway width below 2.5 meters.
 9. **Build Vendor Permit Management Module:** Create light admin portal for station authorities to issue digital spatial permits to registered informal vendors.
 10. **Validate Commercial Model:** Run trial retail zoning analysis at Dukuh Atas TOD area, presenting revenue potential maps to transit commercial teams.

Sprint 11: Multi-City National Transit Hub Expansion

- **Deliverable:** Multi-City Localization & National Dashboard.
- **Key Features:**
 - Pre-configured spatial profiles for top Indonesian transit hubs across Jabodetabek, Surabaya, Bandung, and Medan.
 - National Transit Choke Index leaderboard for the Ministry of Transportation (Kemenhub).
 - Multi-language support and regional paratransit customization (e.g., Angkot vs. Becak vs. Pete-pete).
- **Why this Sprint?:** Scales the platform from a local Jakarta proof-of-concept to a nationwide platform.
- **Tech Stack & Infrastructure:**
 - **Internationalization (i18n):** next-intl (Bahasa Indonesia & English).
 - **Database Partitioning:** PostGIS table partitioning by region (Java, Sumatra, Bali).
- **Tools & Procurement (100% Free Tier):**
 - **Vercel / Cloudflare Pages Free Tier** multi-region edge deployment.
 - **BPS (Badan Pusat Statistik) National Administrative Boundary Open Data.**
- **Step-by-Step Implementation Procedure (10 Steps):**
 1. **Implement next-intl i18n Architecture:** Configure internationalization support for Bahasa Indonesia, English, and regional dialects.
 2. **Ingest BPS National Boundary Datasets:** Import BPS (Badan Pusat Statistik) national administrative boundary GeoJSON into PostGIS.
 3. **Partition PostGIS Spatial Database:** Implement PostGIS table partitioning by geographic region (Java, Sumatra, Bali, Sulawesi).
 4. **Configure Regional Paratransit Models:** Add modular configuration support for local transit types (Angkot in Jakarta/Bandung, Pete-pete in Makassar, Becak in Medan).
 5. **Pre-Configure Spatial Profiles for Top Cities:** Build spatial hub templates for Surabaya (Gubeng), Bandung (Stasiun Hall), Medan (Stasiun Medan), and Makassar.
 6. **Build National Transit Choke Leaderboard:** Create macro-level dashboard for Ministry of Transportation (Kemenhub) ranking nationwide transit exit efficiency.
 7. **Set Up Multi-Region Vercel / Cloudflare Deployments:** Optimize global CDN routing and edge function execution for multi-city low latency.
 8. **Conduct Remote Field Surveyor Onboarding:** Create self-paced digital training guide for regional surveyors using MAPID Form.
 9. **Implement Multi-Region RBAC Authorization:** Extend Clerk / Supabase Auth roles to isolate regional Dishub municipal authorities (e.g., Dishub Surabaya vs. Dishub Bandung).
 10. **Execute Nationwide Beta Launch:** Successfully launch TransitFlow AI across 5 pilot transit hubs in Surabaya, Bandung, and Medan.

Sprint 12: Enterprise API / SDK & Autonomous Traffic Dispatch

- **Deliverable:** Open Developer API & Smart City Autonomous System Integration.
- **Key Features:**
 - REST & GeoJSON API for third-party navigation apps (Google Maps, Moovit, Grab, Gojek) to ingest live station exit choke status.
 - Autonomous traffic light timing integration (triggering longer green lights at exits under severe choke risk).
 - Complete end-to-end platform audit, performance hardening, and enterprise SLAs.
- **Why this Sprint?:** Completes the vision—transforming TransitFlow AI into an indispensable open smart-city infrastructure standard.
- **Tech Stack & Infrastructure:**
 - **API Gateway & Rate Limiting:** Hono / Fastify API Gateway, Upstash Redis Rate Limiter, OpenAPI 3.0 / Swagger.
 - **Smart Traffic Signal Interface:** NTCIP 1202 / SCATS / ATCS (Adaptive Traffic Control System) protocol adapters.
- **Tools & Procurement (100% Free Tier):**
 - **Upstash Redis Free Tier** for rate limiting (10,000 req/day).
 - **GlitchTip / Sentry Open Source Free Tier** for APM error monitoring.
 - **OWASP ZAP** open-source security scanner.
- **Step-by-Step Implementation Procedure (10 Steps):**
 1. **Build Hono / Fastify Enterprise API Gateway:** Create lightweight REST & GeoJSON API gateway for third-party developer integration.
 2. **Implement Upstash Redis Rate Limiting:** Configure token-bucket rate limiting on API endpoints (10,000 req/day free allocation).
 3. **Publish OpenAPI 3.0 / Swagger Documentation:** Generate interactive developer API documentation hosted on Vercel / Cloudflare Pages.
 4. **Develop Navigational API Endpoints:** Expose `/api/v1/hubs/{id}/exit-status` for third-party navigation apps (Google Maps, Moovit, Grab, Gojek).
 5. **Build NTCIP 1202 Traffic Signal Protocol Adapter:** Develop protocol bridge converting VCI choke alerts into NTCIP 1202 standards for Adaptive Traffic Control Systems (ATCS).
 6. **Implement Autonomous Traffic Light Triggering:** Configure automated green-light extension trigger when exit choke exceeds critical threshold ($VCI \geq 85$).
 7. **Integrate Open-Source GlitchTip / Sentry Monitoring:** Deploy free open-source APM error logging and performance tracking.
 8. **Execute OWASP ZAP Automated Penetration Testing:** Run security vulnerability scans and apply security hardening across API endpoints.
 9. **Perform End-to-End SLA Hardening:** Optimize API query caching to guarantee 99.9% uptime and $<50\text{ms}$ GeoJSON payload response times.
 10. **Publish TransitFlow Open SDK:** Release open-source JavaScript/TypeScript SDK (@transitflow/sdk) for smart-city integration.

Master Technology & Zero-Cost Free-Tier Procurement Matrix

Category	Core Technology	Commercial Service	Zero-Cost Free-Tier / MAPID Perk Alternative	Free-Tier Allowance
MAPID Ecosystem	MAPID Form, SINI AI, GEO MAPID, GEO DATA	MAPID Pro Plan	MAPID Free Tier / Top 50 Perk	Provided 100% free by MAPID to Top 50 teams.
Frontend WebGIS	React 19, Next.js 16, Tailwind, MapLibre GL JS	Vercel Pro	Vercel Free Tier / Cloudflare Pages	Unlimited bandwidth free.
Backend & DB	Node.js (TS), PostGIS	AWS RDS PostgreSQL	Supabase Free Tier (PostGIS enabled)	500 MB DB + 50,000 MAU free.
AI / Multimodal	Vision AI, Audio Transcribe	OpenAI GPT-4o / Whisper API	Google Gemini 3.6 Flash Free + Whisper local	15 RPM / 1M TPM free + 100% open-source local Whisper.
Object Detection	Computer Vision / Crowd Count	Commercial Vision API	YOLO26 / Vision Transformers (Open Source)	100% free open-source PyTorch model execution.
Caching & Stream	Redis Pub/Sub, WebSockets	Paid Pusher / Redis Enterprise	Upstash Redis Free + Socket.io on Render	10,000 req/day free + 750 free server hours/mo.
Auth & B2G RBAC	Multi-tenant Auth	Auth0 Enterprise	Clerk / Supabase Auth Free Tier	10,000 monthly active users free.